

PASSPORT MANAGEMENT SYSTEM

Object-Oriented Analysis & Design (OOAD) Technical Report & Architecture Specification

Project:	Passport Management Web Application	Date:	September 2026
Framework:	Java 17 / Spring Boot 3.3.4	Database:	In-Memory H2 (Transactional JPA)
Frontend:	Apple Human Interface Design / HTML5 / JS	Security:	Session-based RBAC Interceptor
Deployment:	Render Cloud PaaS (Dockerized Container)	Repository:	github.com/abelosbert06/passport-management-system

Executive Summary

The Passport Management System is a modern, enterprise-grade multi-tier web application designed to streamline the end-to-end lifecycle of citizen passport operations. Developed strictly following rigorous Object-Oriented Analysis and Design (OOAD) principles, the system models real-world actors, governmental business workflows, and verification requirements associated with public travel document administration.

The application provides distinct, role-segregated interfaces for Citizens (Applicants) and Verification Officers. Applicants register verified demographic profiles, submit regular or expedited (Tatkaal) applications, track processing progress transparently via unforgeable algorithmic tracking numbers, and view digital passport cards. Verification Officers possess a specialized verification desk to audit legal credentials, enter immutable audit commentary, and trigger official passport issuance.

Core System Objectives & Architectural Scope

- 1. Rigorous Domain Modeling:** Encapsulate domain entities (Applicant, PassportApplication, VerificationRecord, PassportRecord, User) with high cohesion and low coupling using Spring Data JPA.
- 2. Role-Based Access Control (RBAC):** Enforce non-repudiable authorization boundaries ensuring applicants cannot access officer queues, and officers cannot tamper with unauthorized submissions.
- 3. Apple-Inspired Human Interface:** Deliver a distraction-free, minimalist user experience eliminating visual noise, gradients, and emojis, complete with system-responsive Dark and Light visual modes.
- 4. Cloud Native Readiness:** Deployable as an immutable, multi-stage Alpine Docker container on cloud platforms (Render, Railway) with automated continuous integration and reverse-proxy session stability.

Document Structure & Page Organization

Page 2	System Requirements & Object-Oriented Analysis (OOAD Actors, Use Cases, State Lifecycle)
Page 3	System Architecture & Design Patterns (Multi-Tier Layers, GoF Patterns, RBAC Matrix)
Page 4	Domain Model & Database Schema (Entities, Cascading Rules, Data Dictionary)
Page 5	Backend Implementation & Core Code Snippets (Application & Verification Services)
Page 6	Security, Interceptors & REST API Specification (Role Interceptor, Full API Catalog)
Page 7	Frontend Architecture & UI Design System (Apple HIG, Dark Mode Architecture, State Sync)
Page 8	Automated Testing, Containerization & Cloud Deployment (Integration Tests, Docker, Render)

1. System Requirements & Object-Oriented Analysis (OOAD)

A formal Object-Oriented Analysis was performed to translate unstructured administrative problem statements into precise functional boundaries, actor responsibilities, and deterministic domain transitions.

1.1 Functional Requirements (FR)

- FR-1 (Applicant Registration & Profile):** Citizens must register unique credentials tied to verified demographic attributes: full legal name, email, phone number, birth date, and residential address.
- FR-2 (Application Submission):** Logged-in applicants must submit passport requests selecting between REGULAR and TATKAAL processing types, supplying valid national identity proof numbers.
- FR-3 (Automated Tracking Number Synthesis):** Every submission must instantaneously receive an unforgeable, human-readable tracking identifier formatted as PASS-YYYY-[8-CHAR-HEX].
- FR-4 (Officer Verification Terminal):** Authorized verification officers must view pending applicant queues, examine submitted credentials, and commit immutable audit records with explicit PASSED or FAILED decisions and commentary.
- FR-5 (Official Passport Issuance):** Upon verification pass, officers must execute official passport synthesis, generating an official passport number (P + 7 numeric digits) valid for exactly ten calendar years.
- FR-6 (Public Status Tracking):** Any stakeholder possessing a tracking code must be able to query current application status, officer notes, and issuance dates without privilege escalation.

1.2 Non-Functional Requirements (NFR)

- NFR-1 (Role Boundary Isolation):** Enforce strict server-side HTTP session interceptor checks preventing cross-tenant privilege tampering between citizens and verification staff.
- NFR-2 (Transactional Data Consistency):** All domain transitions (verification and issuance) must be executed in atomic database transactions to eliminate race conditions and partial writes.
- NFR-3 (Responsive Visual Fidelity):** Provide seamless accessibility across mobile, tablet, and high-DPI desktop viewports adhering to Apple Human Interface Guidelines and WCAG contrast standards.
- NFR-4 (High Availability & Rapid Bootstrapping):** Containerized initialization cold-start under 3 seconds with zero external database dependencies for assignment reproducibility.

1.3 Stakeholder & Actor Identification

Actor	Role Description	Primary System Interactions
Applicant (Citizen)	External end-user requesting travel documentation.	- Register personal profile - Submit regular or tatkaal application - Monitor review progress & view issued passport pass
Verification Officer (Government Staff)	Internal credentialed officer executing identity audit & legal checks.	- Access administrative review queue - Audit citizen documentation & enter remarks - Grant approval / rejection & issue official passport
Public Auditor (General User)	Anonymous entity checking validity of an application.	- Query tracking endpoint with tracking ID - Validate status without accessing personal data

1.4 Passport Application Lifecycle State Machine

The state lifecycle strictly mandates a unidirectional progression: an application enters as **SUBMITTED**, progresses to **UNDER_REVIEW** during evaluation, transitions to either **APPROVED** or **REJECTED**, and concludes at **ISSUED** upon physical passport generation. Out-of-sequence calls throw an `IllegalStateException`.



2. System Architecture & Design Patterns

The application is architected according to the classic Clean Architecture and Multi-Tier Enterprise pattern. High separation of concerns ensures presentation, domain business rules, authentication guards, and data persistence remain completely decoupled.

2.1 Layered Architecture Overview

Layer	Technology / Component	Architectural Responsibility
Presentation Layer	HTML5 / Modern JS / CSS3 (Apple HIG)	Renders reactive client dashboards, segmented navigation, and dark mode theming.
Security / Interceptor	Spring HandlerInterceptor	Intercepts HTTP calls, validates session authentication, and enforces role boundaries.
Web / Controller Layer	Spring MVC @RestController	Exposes RESTful endpoints, unmarshals JSON payloads, and handles Bean Validation.
Service / Domain Layer	Spring @Service with Transactions	Executes core business rules: state machines, ID generators, and verification outcomes.
Data Access Layer	Spring Data JPA Repositories	Abstracts database dialect, generates type-safe queries, and manages object lifecycles.
Persistence Layer	In-Memory H2 Relational Database	ACID transactional relational persistence, automatically seeded with baseline test data.

2.2 Object-Oriented & Gang of Four (GoF) Design Patterns

- 1. Repository Pattern:** Interfaces like `PassportApplicationRepository` extend `JpaRepository`, completely isolating data-access mechanisms from business services. This adheres to the Dependency Inversion Principle (DIP).
- 2. Interceptor Pattern:** `RoleAuthInterceptor` implements Spring's `HandlerInterceptor` to provide aspect-oriented cross-cutting security checks without polluting controller handler methods.
- 3. Data Transfer Object (DTO) Pattern:** Dedicated request/response objects (`ApplicationSubmissionRequest`, `LoginResponse`, `TrackingResponse`) prevent over-posting vulnerabilities and decouple API contracts from internal JPA models.
- 4. Factory & Static Synthesizer Pattern:** Entity conversion and projection factories (e.g., `TrackingResponse.fromEntity(...)` and `LoginResponse.fromUser(...)`) encapsulate object instantiation logic in single, highly cohesive methods.
- 5. Inversion of Control (IoC) & Dependency Injection:** Strict constructor-based injection is utilized across all controllers and services, guaranteeing deterministic unit-testability without reflection magic.

2.3 Role-Based Access Control (RBAC) Permission Matrix

Endpoint URI Pattern	HTTP	Permitted Role	Security Enforcement Action
/api/auth/login, /register	POST	PUBLIC	Open access; authenticates and registers sessions.
/api/auth/me, /logout	GET/POST	ANY AUTH	Requires valid session cookie; returns active user or clears session.
/api/applications	POST	APPLICANT	Enforces Role.APPLICANT; automatically injects session applicantId.
/api/applications/my	GET	APPLICANT	Restricted to authenticated applicant; returns own dossiers only.
/api/applications/track/**	GET	PUBLIC	Open access; returns sanitized tracking projection.
/api/officer/**	ALL	OFFICER	Strictly requires Role.OFFICER; returns 403 Forbidden for others.

3. Domain Model & Database Schema

The relational database schema is normalized and mapped via Jakarta Persistence (JPA). Domain entities model real-world passport entities while maintaining strict relational integrity and cascading constraints.

3.1 Entity Relationship Diagram (ERD) Specifications

- **Applicant (1) <---> (N) PassportApplication:** One citizen can submit multiple passport applications across time (e.g. renewal or tatkaal).
- **PassportApplication (1) <---> (1) VerificationRecord:** Each application maintains an optional 1-to-1 association with its official audit record once inspected by an officer.
- **PassportApplication (1) <---> (1) PassportRecord:** An issued application links to exactly one official passport record containing the unique passport number.
- **User (1) <---> (1) Applicant:** Applicant user accounts are linked via a 1-to-1 foreign key to their citizen profile. Officer accounts hold Role.OFFICER with an officer name string.

3.2 Technical Deep Dive: Overcoming Hibernate Cascade Conflicts

During verification and issuance flows, child entities (VerificationRecord and PassportRecord) are instantiated, validated, and saved. When using blanket CascadeType.ALL or CascadeType.PERSIST on PassportApplication, Hibernate 6 threw a PersistentObjectException: detached entity passed to persist because child entities already assigned generated IDs during their independent repository operations were re-persisted by cascade traversal.

Architectural Resolution: Replaced CascadeType.ALL with explicit selective cascading: {CascadeType.MERGE, CascadeType.REFRESH, CascadeType.REMOVE}. This ensures child records are merged without re-invoking persist triggers, guaranteeing rock-solid transaction commits.

3.3 Comprehensive Data Dictionary

Entity	Column Name	Data Type	Constraints / Description
Applicant	id	BIGINT	Primary Key (AUTO_INCREMENT)
	full_name	VARCHAR(255)	Not Null; Citizen legal full name
	email	VARCHAR(255)	Not Null; Unique index
	phone_number	VARCHAR(50)	Not Null; Contact phone
	date_of_birth	DATE	Not Null; Citizen birth date
	address	VARCHAR(255)	Not Null; Residential address
Passport Application	id	BIGINT	Primary Key (AUTO_INCREMENT)
	tracking_number	VARCHAR(64)	Not Null; Unique tracking code
	applicant_id	BIGINT	Foreign Key -> Applicant(id)
	passport_type	VARCHAR(20)	Enum: REGULAR, TATKAAL
	status	VARCHAR(20)	Enum: SUBMITTED, UNDER_REVIEW, etc.
	id_proof_number	VARCHAR(100)	National ID proof document #
Verification Record	submitted_at	TIMESTAMP	Creation timestamp (Auto)
	id	BIGINT	Primary Key (AUTO_INCREMENT)
	officer_name	VARCHAR(255)	Name of verifying officer
	outcome	VARCHAR(20)	Enum: PASSED, FAILED
	remarks	VARCHAR(500)	Audit notes and remarks
Passport Record	verified_at	TIMESTAMP	Timestamp of officer sign-off
	id	BIGINT	Primary Key (AUTO_INCREMENT)
	passport_number	VARCHAR(30)	Unique (P + 7 numeric digits)
	issue_date	DATE	Official date of issuance
	expiry_date	DATE	Issue date + 10 calendar years
User	status	VARCHAR(20)	Active status indicator
	id	BIGINT	Primary Key (AUTO_INCREMENT)
	username	VARCHAR(100)	Unique login username
	password	VARCHAR(255)	Account password credential
	role	VARCHAR(20)	Enum: APPLICANT, OFFICER
	applicant_id	BIGINT	Nullable Foreign Key -> Applicant

4. Backend Implementation & Core Code Snippets

Business operations are encapsulated within transactional service beans. Below are the canonical implementations for application submission, unique tracking generation, verification audits, and passport number sequencing.

4.1 PassportApplicationService.java — Submission & Tracking Synthesis

```
@Service @Transactional
public class PassportApplicationService {
    private final PassportApplicationRepository applicationRepository;
    private final ApplicantRepository applicantRepository;

    public PassportApplication submitApplication(ApplicationSubmissionRequest request) {
        Applicant applicant = applicantRepository.findById(request.getApplicantId())
            .orElseThrow(() -> new IllegalArgumentException("Applicant not found: " + request.getApplicantId()));

        // Synthesis of unforgeable algorithmic tracking identifier (e.g., PASS-2026-A1B2C3D4)
        String trackingNumber = "PASS-" + LocalDateTime.now().getYear() + "-" +
            UUID.randomUUID().toString().substring(0, 8).toUpperCase();

        PassportApplication application = new PassportApplication(
            trackingNumber, applicant, request.getPassportType(),
            ApplicationStatus.SUBMITTED, request.getIdProofNumber(), LocalDateTime.now()
        );
        return applicationRepository.save(application);
    }
}
```

4.2 VerificationService.java — Officer Audit & Passport Issuance Logic

```
@Service @Transactional
public class VerificationService {
    private final PassportApplicationRepository appRepo;
    private final VerificationRecordRepository verifyRepo;
    private final PassportRecordRepository passportRepo;

    public PassportApplication verifyApplication(Long id, VerificationRequest request) {
        PassportApplication app = appRepo.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Application not found: " + id));

        VerificationRecord record = new VerificationRecord(
            request.getOfficerName(), request.getOutcome(), request.getRemarks(), LocalDateTime.now()
        );
        record = verifyRepo.save(record);
        app.setVerificationRecord(record);
        app.setStatus(request.getOutcome() == VerificationOutcome.PASSED ? ApplicationStatus.APPROVED : ApplicationStatus.REJECTED);
        return appRepo.save(app);
    }

    public PassportApplication issuePassport(Long applicationId) {
        PassportApplication app = appRepo.findById(applicationId)
            .orElseThrow(() -> new IllegalArgumentException("Application not found"));
        if (app.getStatus() != ApplicationStatus.APPROVED) {
            throw new IllegalStateException("Only APPROVED applications can be issued a passport.");
        }

        String passportNum = "P" + String.format("%07d", new Random().nextInt(10_000_000));
        LocalDate issueDate = LocalDate.now();
        PassportRecord passport = new PassportRecord(passportNum, issueDate, issueDate.plusYears(10), "ACTIVE");
        app.setPassportRecord(passportRepo.save(passport));
        app.setStatus(ApplicationStatus.ISSUED);
        return appRepo.save(app);
    }
}
```

4.3 Business Rules & Concurrency Highlights

- **Declarative Transactional Demarcation:** @Transactional guarantees atomic commits. If database writing fails during passport linking, the entire transaction rolls back cleanly.
- **State Invariant Enforcement:** Issuing an unapproved or rejected application throws an IllegalStateException, preventing illegal state transitions at the Java bytecode level.
- **Decoupled Domain Auditing:** Verifications record officer name and timestamp independently, preserving audit trails even if applications undergo subsequent re-verifications.

5. Security, Interceptors & REST API Specification

Application security is handled via a lightweight, zero-dependency Spring HandlerInterceptor mechanism. The interceptor interrogates the active HTTP session before requests reach controllers.

5.1 RoleAuthInterceptor.java — Declarative Role Boundary Guard

```
@Component
public class RoleAuthInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) throws Exception {
        String uri = req.getRequestURI();
        HttpSession session = req.getSession(false);
        LoginResponse user = session != null ? (LoginResponse) session.getAttribute(AuthController.SESSION_USER) : null;

        if (uri.startsWith("/api/officer")) {
            if (user == null) { sendError(res, 401, "Authentication required as Officer."); return false; }
            if (user.getRole() != Role.OFFICER) { sendError(res, 403, "Access denied: Officer role required."); return false; }
        }
        if (uri.equals("/api/applications") && "POST".equalsIgnoreCase(req.getMethod())) {
            if (user == null || user.getRole() != Role.APPLICANT) {
                sendError(res, 401, "Please log in as an Applicant to submit."); return false;
            }
        }
        return true;
    }
}
```

5.2 Global Exception Handling Architecture

A centralized @RestControllerAdvice catches all unchecked runtime exceptions (IllegalArgumentException, IllegalStateException, and MethodArgumentNotValidException), converting them into uniform JSON responses containing RFC-compliant HTTP status codes and actionable error messages.

5.3 Comprehensive REST API Catalog

HTTP	Endpoint URI	Role	Request Body / Params	Response
POST	/api/auth/login	PUBLIC	{username, password}	200 OK + Session Cookie
POST	/api/auth/register	PUBLIC	{username, password, fullName, email, ...}	201 Created (User)
GET	/api/auth/me	AUTH	None (Reads JSESSIONID)	200 OK (User Profile)
POST	/api/auth/logout	AUTH	None (Invalidates Session)	204 No Content
POST	/api/applications	APPLICANT	{passportType, idProofNumber}	201 Created (App Entity)
GET	/api/applications/my	APPLICANT	None (Filtered by Session ID)	200 OK (List of Apps)
GET	/api/applications/track/{trk}	PUBLIC	Path Variable: trackingNumber	200 OK (Tracking DTO)
GET	/api/officer/applications	OFFICER	Query Param: ?status=SUBMITTED	200 OK (Filtered Queue)
POST	/api/officer/applications/{id}/verify	OFFICER	{officerName, outcome, remarks}	200 OK (Updated App)
POST	/api/officer/applications/{id}/issue	OFFICER	Path Variable: applicationId	200 OK (Issued App + Pass)

6. Frontend Architecture & User Interface Design

The client interface strictly embodies Apple Human Interface Guidelines (HIG). It completely eschews visual distractions, multi-color gradients, and emojis in favor of immaculate typography, translucent frosted surfaces, precise hairline dividers, and fluid micro-interactions.

6.1 Apple Human Interface Design System Tokens

- **Translucent Frosted Titlebar:** Styled with backdrop-filter: saturate(180%) blur(20px) and a subtle 0.5px hairline border separating the navigation bar from canvas content.
- **Apple Segmented Controls:** Tab bars utilize an enclosed pill-shaped container (.apple-segmented) with a floating active indicator and smooth transition physics.
- **Apple Wallet-Style Digital Passport Card:** Issued passports render in a dark metallic card chassis featuring the national passport number, validity stamps, and active security badges.
- **macOS Profile Navigation Chip:** Replaced cluttered pill tags with a refined Apple profile item: a circular avatar with user initials, two-tier bold/muted typography, and a subtle icon sign-out button.

6.2 Dynamic Light & Dark Mode Theming Architecture

A full-system dark mode is implemented via scoped CSS custom variables and Bootstrap 5.3's data-bs-theme engine. The interface automatically checks localStorage or queries the operating system's prefers-color-scheme media query.

```
/* Apple Design System Color Variable Mapping */
:root, [data-theme="light"] {
  --apple-bg: #f5f5f7;      --apple-card: #ffffff;
  --apple-text: #1d1d1f;    --apple-secondary: #86868b;
  --apple-hairline: rgba(0, 0, 0, 0.12);
  --apple-blue: #0071e3;    --apple-blue-hover: #0077ed;
}
[data-theme="dark"] {
  --apple-bg: #000000;      --apple-card: #1c1c1e;
  --apple-text: #f5f5f7;    --apple-secondary: #8e8e93;
  --apple-hairline: rgba(255, 255, 255, 0.12);
  --apple-blue: #0a84ff;    --apple-blue-hover: #409cff;
}
```

6.3 Role-Segregated Dashboards & Workflow

Dashboard Portal	Active Component	Functional Workflow Details
Applicant Portal	My Applications Feed • Real-time card list • Status indicators • Digital wallet passport	Displays all historical and active submissions linked to the logged-in applicant. Shows officer review notes upon completion and active passport passes.
Applicant Portal	Application Form • Regular / Tatkaal selection • National ID validation	Streamlined single-action submission form. Automatically passes applicantId from session context and displays instant confirmation with tracking number.
Officer Portal	Verification Desk • Status filter dropdown • Document audit modal • Direct issuance button	Queue view showing all citizen applications across the country. Officers inspect submissions, enter binding audit remarks, and click to execute immediate passport generation.

7. Testing, Containerization & Cloud Deployment

Quality assurance is validated via automated integration tests covering the full lifecycle. For production delivery, the service is containerized using multi-stage Docker builds and hosted on cloud infrastructure.

7.1 Automated Integration Testing Suite (JUnit 5 / Spring Boot Test)

```
@SpringBootTest
class PassportApplicationTests {
    @Autowired private PassportApplicationService applicationService;
    @Autowired private VerificationService verificationService;
    @Autowired private ApplicantRepository applicantRepository;

    @Test
    void testCompletePassportLifecycle() {
        Applicant applicant = applicantRepository.save(
            new Applicant("Alice Wonderland", "alice@gov.test", "+1234567890", LocalDate.of(1998, 5, 20), "10 Downing St")
        );
        PassportApplication app = applicationService.submitApplication(
            new ApplicationSubmissionRequest(applicant.getId(), PassportType.REGULAR, "PROOF-XYZ-99")
        );
        assertEquals(ApplicationStatus.SUBMITTED, app.getStatus());

        PassportApplication verified = verificationService.verifyApplication(
            app.getId(), new VerificationRequest("Officer Brown", VerificationOutcome.PASSED, "Identity verified")
        );
        assertEquals(ApplicationStatus.APPROVED, verified.getStatus());

        PassportApplication issued = verificationService.issuePassport(app.getId());
        assertEquals(ApplicationStatus.ISSUED, issued.getStatus());
        assertNotNull(issued.getPassportRecord().getPassportNumber());
    }
}
```

7.2 Multi-Stage Dockerfile Architecture

```
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml . && RUN mvn dependency:go-offline -B
COPY src ./src && RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/passport-management-*.jar app.jar
ENV PORT=8080
EXPOSE 8080
CMD ["sh", "-c", "java -Dserver.port=${PORT} -jar app.jar"]
```

7.3 Production Cloud Deployment on Render

- **Dynamic Port Binding:** Configured `server.port=${PORT:8080}` to seamlessly bind to Render's dynamic ports.
- **Reverse Proxy Support:** Configured `server.forward-headers-strategy=framework` preserving HTTPS cookies across cloud load balancers.
- **Automated Continuous Deployment:** Commits to main at github.com/abelosbert06/passport-management-system trigger automated builds.

8. Conclusion & Architectural Summary

The Passport Management System exemplifies modern software engineering principles: combining disciplined Object-Oriented Analysis and Design (OOAD), clean multi-tiered architecture, robust role-based session authorization, an Apple Human Interface frontend, and automated cloud delivery. The result is a secure, performant, and maintainable enterprise software artifact fully validated for academic and operational submission.